

Calcolo Parallelo e Distribuito

I parametri di valutazione di un algoritmo parallelo

Docente: Prof. L. Marcellino

Tutor: Proff. P. De Luca – A. Esposito

Previously on… CPD

Previously on… CPD

- **Classificazione di Flynn**
  - Temporale → SIDS/MISD
  - Spaziale → SIMD
  - Asincrono → MIMD (DM-SM)
- **Macchine multicore e libreria openMP per la programmazione parallela in questo ambiente**
- **I° nucleo computazionale: somma tra due vettori di lunghezza N – decomposizione del dominio (algoritmi full parallel)**
- **II° nucleo computazionale: somma di N numeri – decomposizione del dominio – collezione dei risultati (1-2 strategia)**

Ho le strategie per la parallelizzazione, ho gli strumenti (macchina multicore e libreria openMP)…



…vale la pena implementare in un software la strategia di parallelizzazione pensata per l'algoritmo?

Analisi algoritmo parallelo

Analisi algoritmo parallelo

Valutare l'efficienza di un algoritmo in ambiente di calcolo parallelo



Cosa si intende per

EFFICIENZA DI UN ALGORITMO ?

Efficienza di un **algoritmo sequenziale**

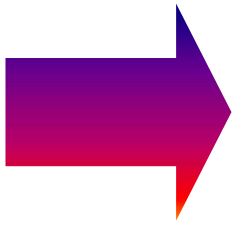
- COMPLESSITA' COMPUTAZIONALE T(N)
  Numero di operazioni eseguite dall'algoritmo

- COMPLESSITA' SPAZIALE S(N)
  Numero di variabili utilizzate dall'algoritmo

Di cosa stiamo parlando?

$$\tau = k \cdot T(N) \cdot \mu$$

μ = tempo di esecuzione di 1 op. f.p.

$T(N) = T_1(N)$  $T_1(N)$ = complessità computazionale 1 processore

Tempo di esecuzione software sequenziale

$$\tau_1 = T_1(N) \cdot K \ \mu$$

t_{calc} = μ tempo di esecuzione di una addizione

somma di due vettori di dimensione N

Su un calcolatore monoprocessore la somma è calcolata eseguendo le *N* addizioni una per volta secondo un ordine prestabilito

$c_0 := a_0 + b_0$
$c_1 := a_1 + b_1$
...
...
$c_{N-1} := a_{N-1} + b_{N-1}$

Esempio:

calcolo della **somma di vettori di dimensione *N=16***

ALGORITMO SEQUENZIALE

numero di addizioni = 16

passi temporali = 16



Passi temporali i=0,…,15

complessità computazionale **T(N)=N addizioni**

In un algoritmo sequenziale

Il numero complessivo di operazioni

è uguale al

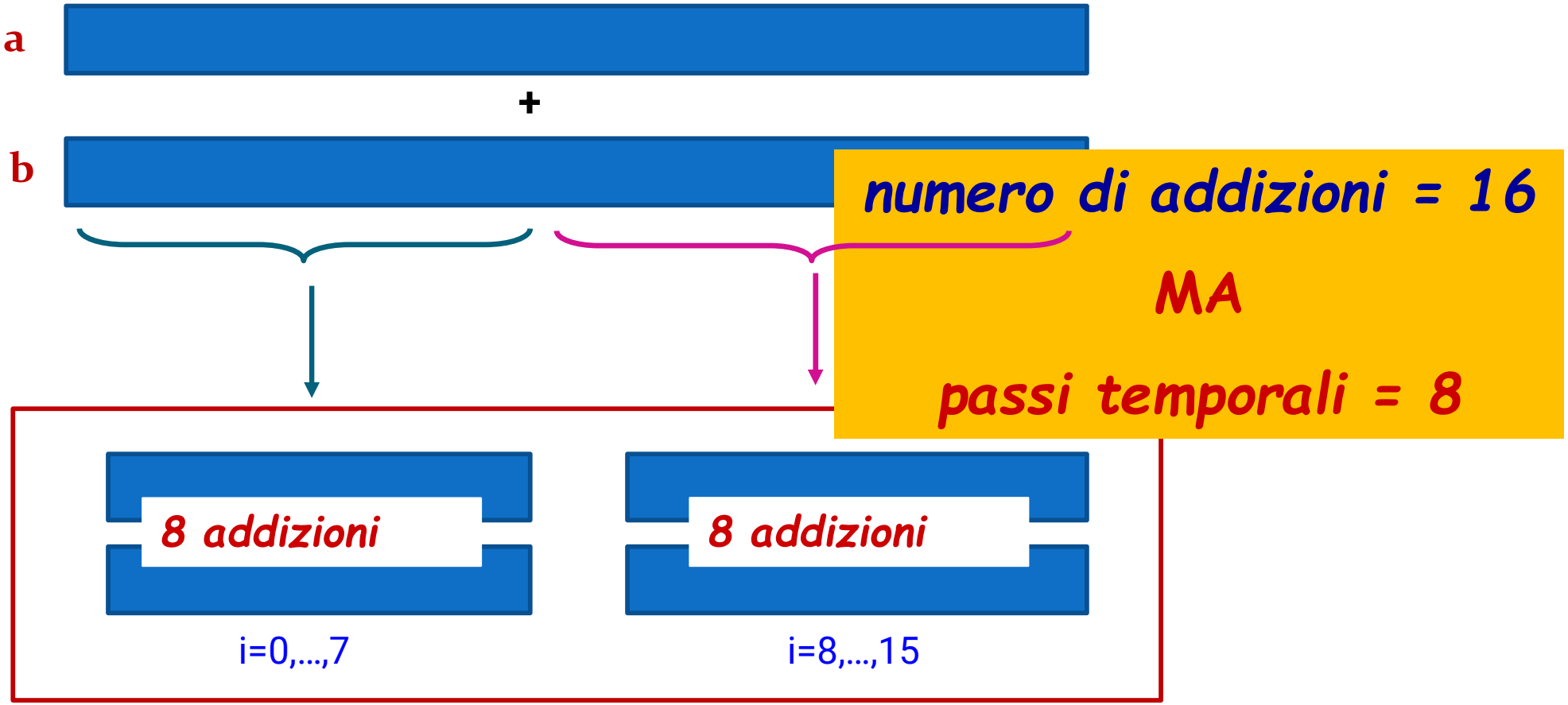
numero dei passi temporali

Esempio:

calcolo della **somma di vettori di dimensione N=16**

ALGORITMO PARALLELO

2 core

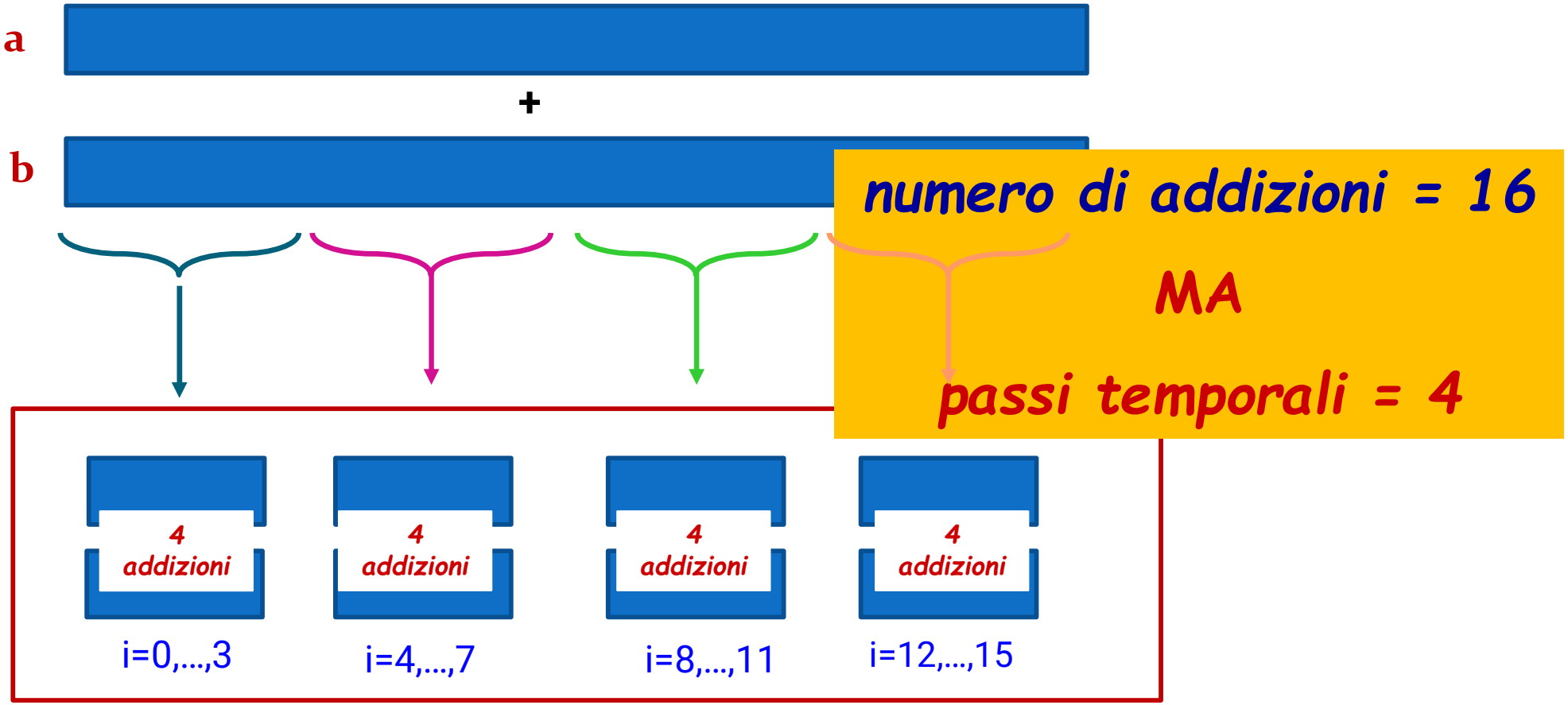


Esempio:

calcolo della **somma di vettori di dimensione *N=16***

ALGORITMO PARALLELO

4 core



numero di addizioni = 16

MA

passi temporali = 4

Nell'algoritmo parallelo della somma

Il numero delle operazioni

non è uguale

al numero dei passi temporali

Infatti …

Un calcolatore parallelo è in grado di eseguire più operazioni

concorrentemente

(allo stesso passo temporale)



Il tempo di esecuzione non è proporzionale alla complessità di tempo
(ovvero non dipende soltanto dal numero di operazioni fl. p. effettuate)



La complessità di tempo non è adatta a misurare l'efficienza di un algoritmo parallelo

… e allora

Che cosa si intende per

efficienza

di un algoritmo

in ambiente parallelo?

In generale

Con p core ci aspettiamo che T_1 sia p volte T_p

ovvero

ci aspettiamo di ridurre di p volte il tempo di esecuzione

In questo caso le cose vanno proprio così…

Misuriamo di quanto si riduce il tempo di esecuzione su p processori

rispetto al tempo di esecuzione

su 1 processore…

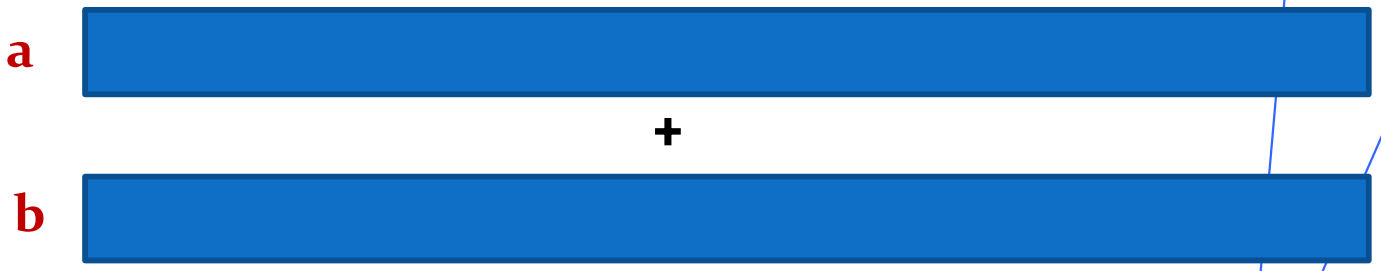
Esempio:

calcolo della **somma di vettori di dimensione *N=16***

ALGORITMO SEQUENZIALE

numero di addizioni = 16

passi temporali = 16



Passi temporali i=0,…,15

$T_1(16)=16$

complessità computazionale

T(N)=N addizioni

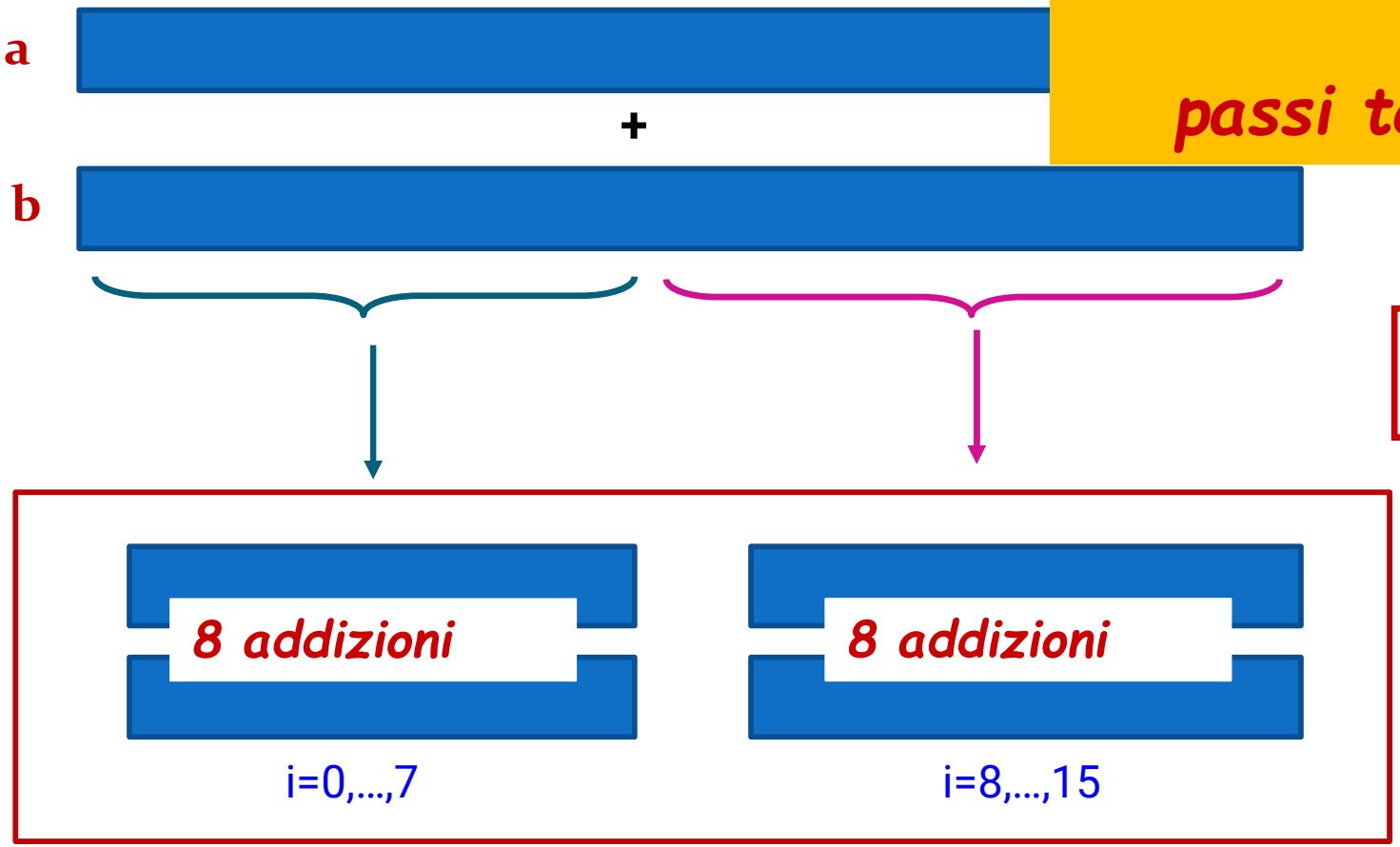
18

Esempio:

calcolo della **somma di vettori di dimensione N=16**

ALGORITMO PARALLELO

2 core



numero di addizioni = 16

MA

passi temporali = 8

$T_2(16)=8$

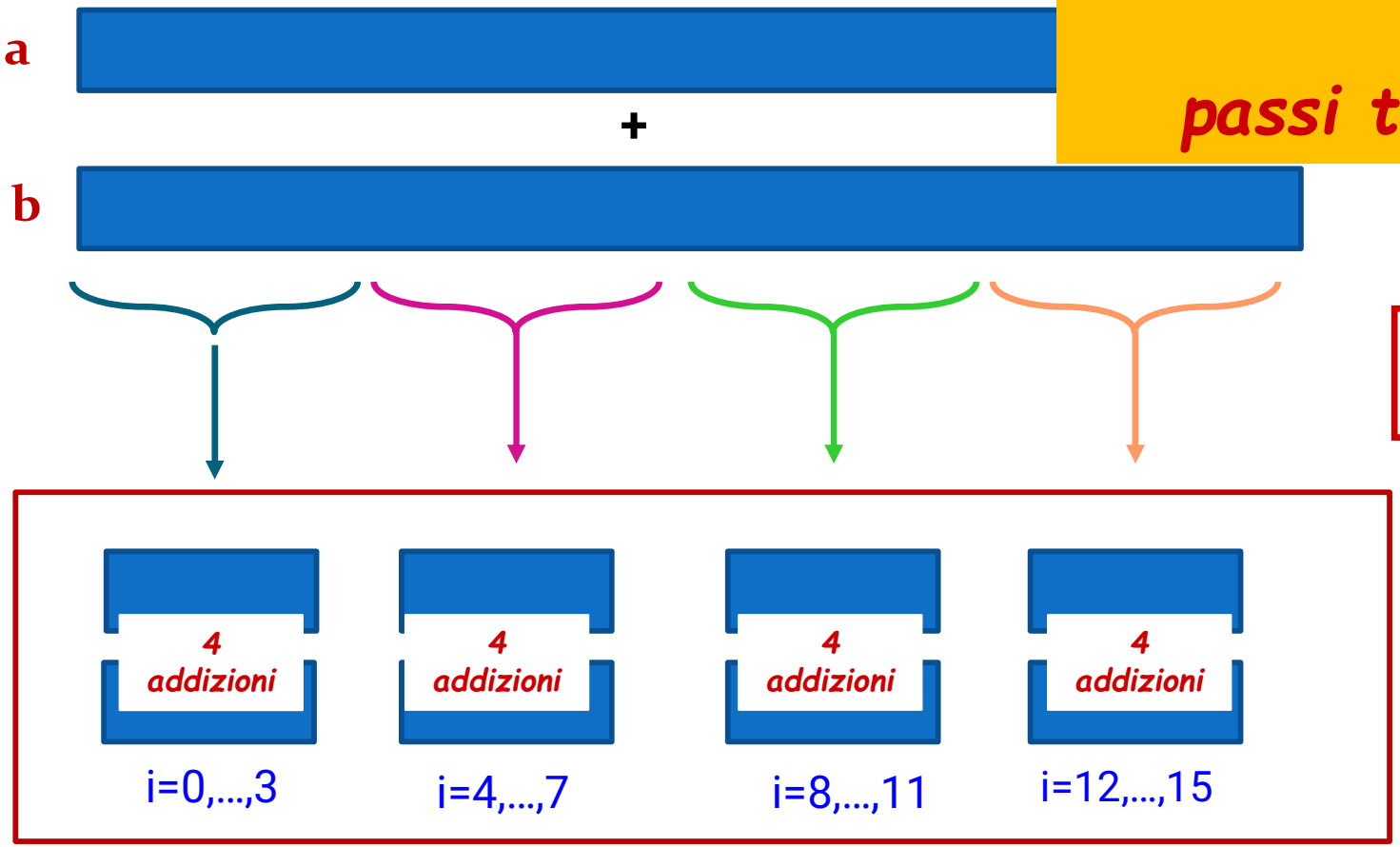
complessità computazionale 2 processori

Esempio:

calcolo della **somma di vettori di dimensione *N=16***

ALGORITMO PARALLELO

4 core



numero di addizioni = 16

MA

passi temporali = 4

$T_4(16)=4$

complessità computazionale 4 processori

Attenzione:

$$\tau = k \cdot T_p(N) \cdot \mu$$

μ = tempo di esecuzione di 1 op. f.p.

$T_p(N)$ → $T_p(N)$= complessità computazionale

p processori

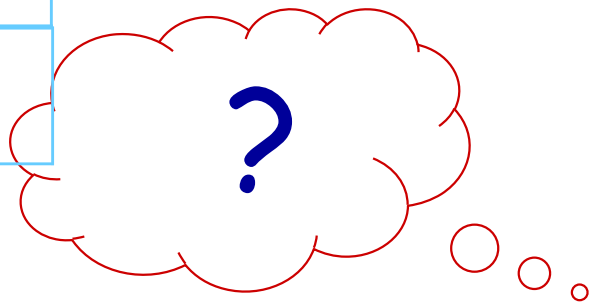
Tempo di esecuzione software parallelo

$\tau_p = T_p(N) \cdot K \ \mu$

t_{calc}= μ tempo di esecuzione di una addizione

In sintesi…

p	$T_p(16)$
1	16
2	8
4	4
8	2
p	?



In generale quanto vale T_p?

In generale: calcolo di $T_p(N)$

ALGORITMO PARALLELO della somma di due vettori di dimensione *N* - posto p processori

p=1 $T_1=16$

p=2 $T_2=8$

p=4 $T_4=4$

p=8 $T_8=2$

.....

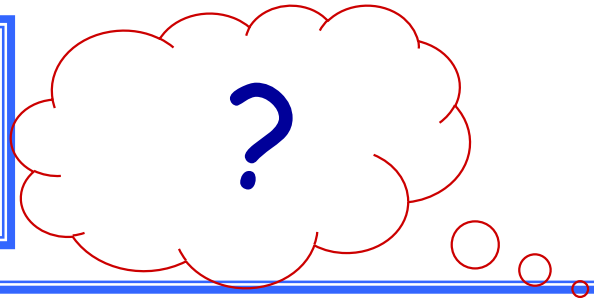
} ***N = 16***

$$T_p(N)= (N/p)$$

Domanda…

p	T_p
1	16
2	8
4	4
8	2

Qual è l'algoritmo che impiega meno tempo?



Quanto è più veloce di quello sequenziale?

Esempio: somma di due vettori di dim N=16

p	T_p	T_1/T_p
1	16	1.00
2	8	2
4	4	4
8	2	8

Maggiore riduzione del tempo ovvero maggiore aumento della velocità

L'algoritmo su 8 processori è il più veloce
E' più veloce di 8 volte di quello su 1 processore

Speed-up

Si definisce il rapporto T_1 su T_p

$$S_p = \frac{T_1}{T_p}$$

Lo speed up misura la riduzione del tempo di esecuzione rispetto all'algoritmo su 1 core

$$S_p < p$$

SPEEDUP IDEALE

$$S_p^{ideale} = p$$

Esempio:

calcolo della **somma di due vettori di dimensione N**

L'algoritmo parallelo
ha speedup ideale!

SPEEDUP IDEALE

$$S_p^{ideale} = p$$

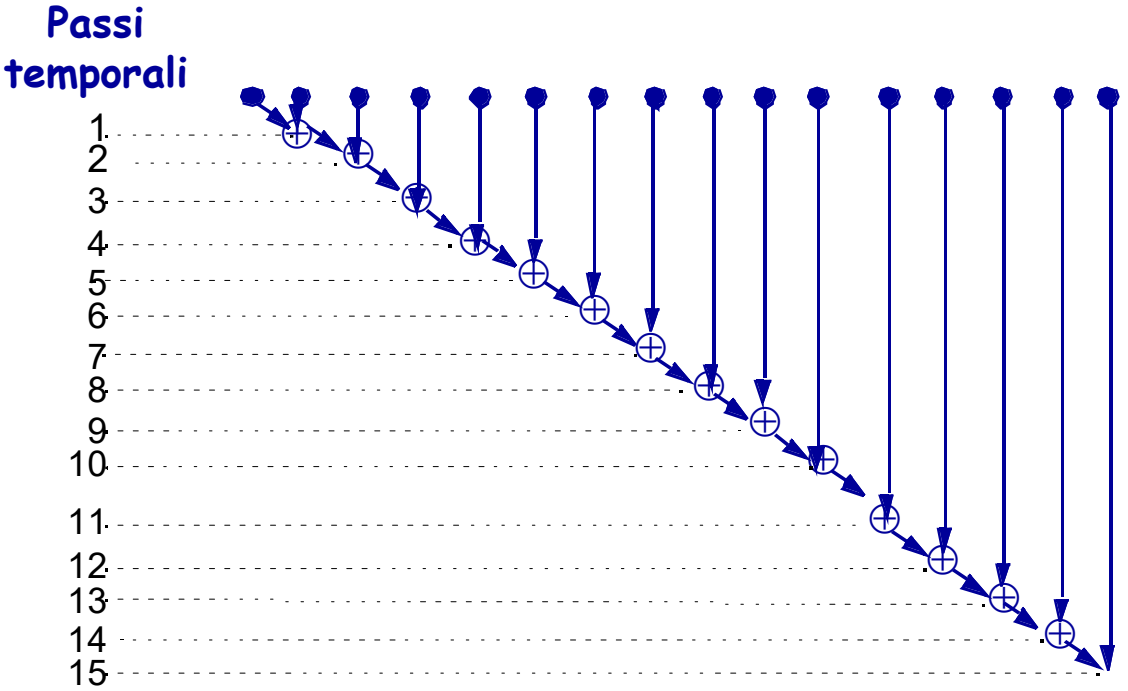


Gli algoritmi full parallel hanno tutti speedup ideale!

Purtroppo non tutti i problemi possono essere risolti con algoritmi completamente parallelizzabili

Esempio: calcolo della somma di N=16 numeri

ALGORITMO SEQUENZIALE



numero di addizioni = 15

passi temporali = 15

Complessità Computazionale

T(N)=N-1 addizioni

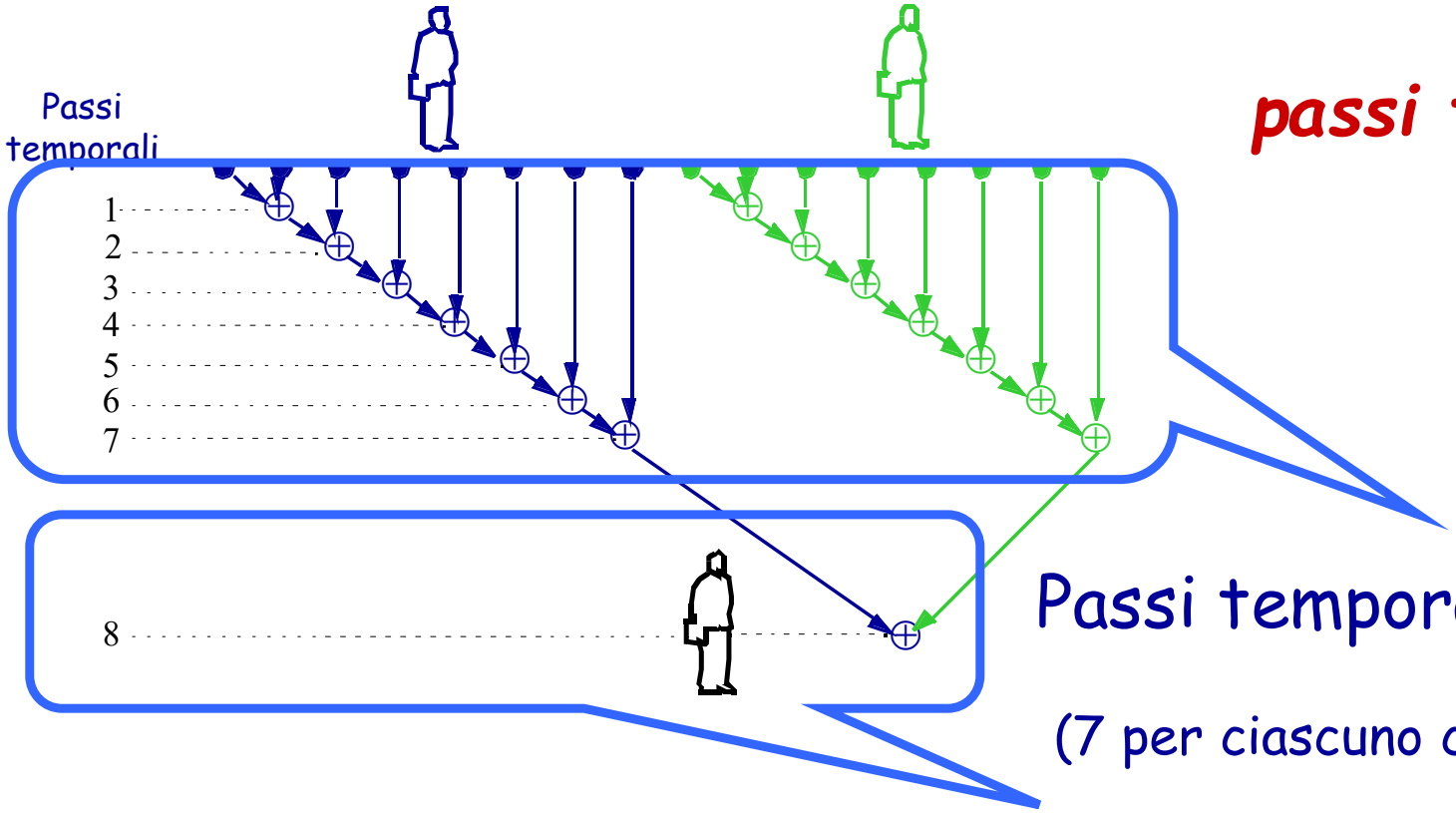
Esempio: calcolo della somma di N=16 numeri

ALGORITMO PARALLELO

numero di addizioni = 15

MA

passi temporali = 8



Passi temporali 1-7: 14 addizioni

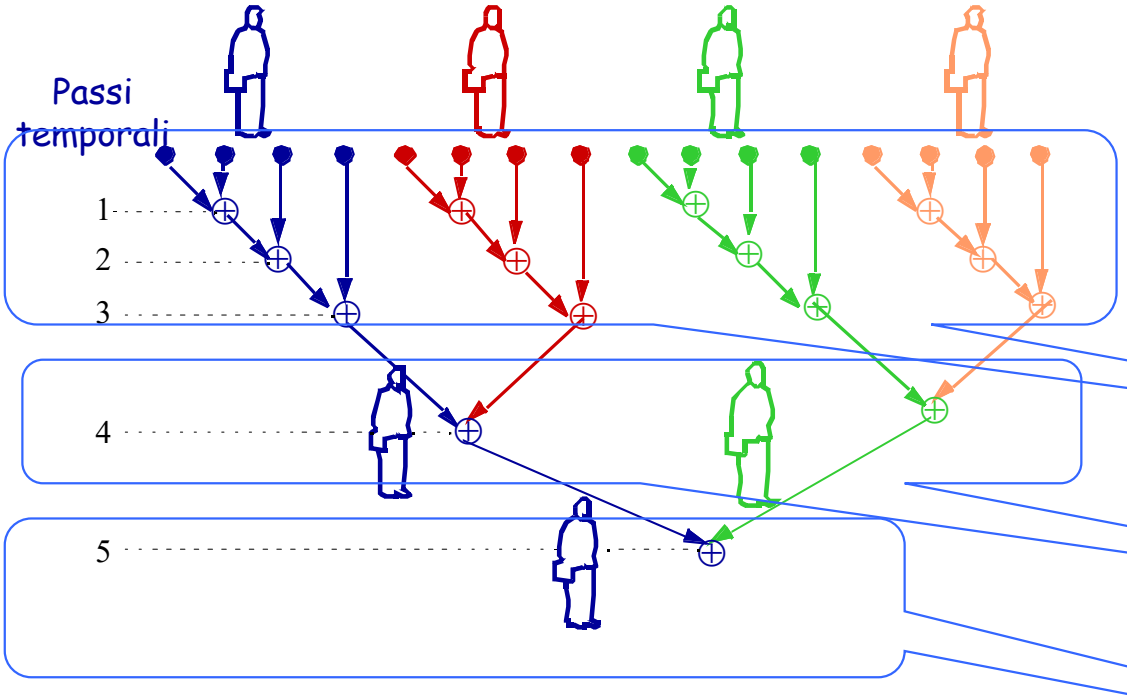
(7 per ciascuno core)

- Passo temporale 8: 1 addizione

Esempio: calcolo della somma di N=16 numeri

ALGORITMO PARALLELO

II strategia



numero di addizioni = 15

MA

passi temporali = 5

Passi temporali 1-3: 12 addizioni

Passo temporale 4: 2 addizioni

Passo temporale 5: 1 addizione

In realtà le cose non vanno proprio così…

Misuriamo di quanto si riduce il tempo di esecuzione su p processori

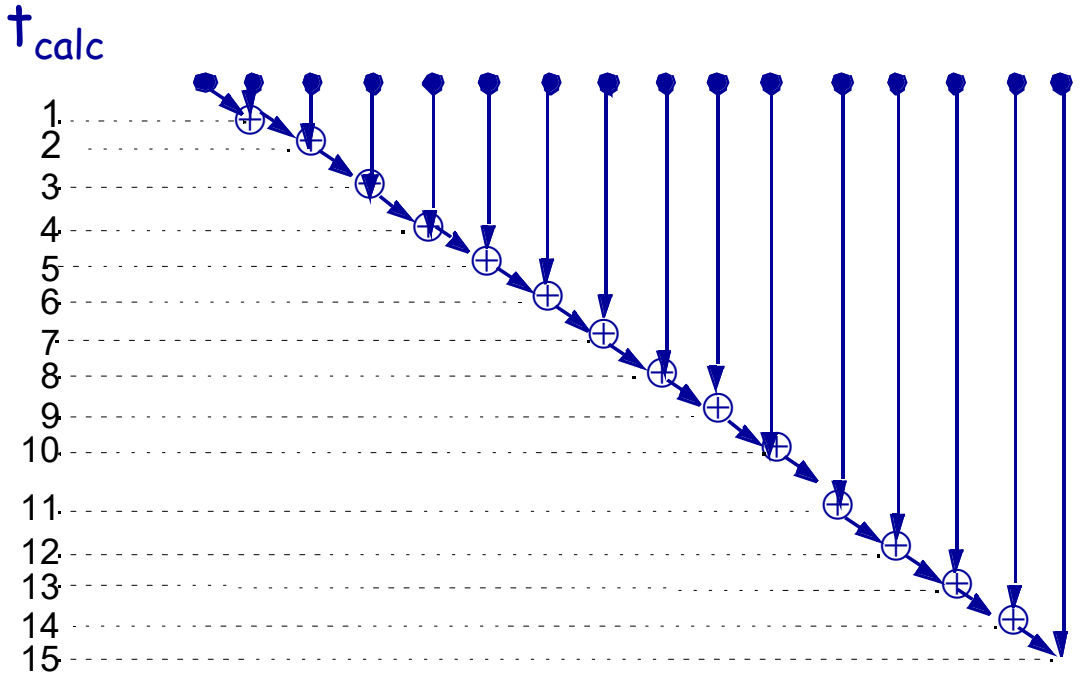
rispetto al tempo di esecuzione

su 1 processore…

Esempio: calcolo della somma di N=16 numeri

ALGORITMO SEQUENZIALE

p=1



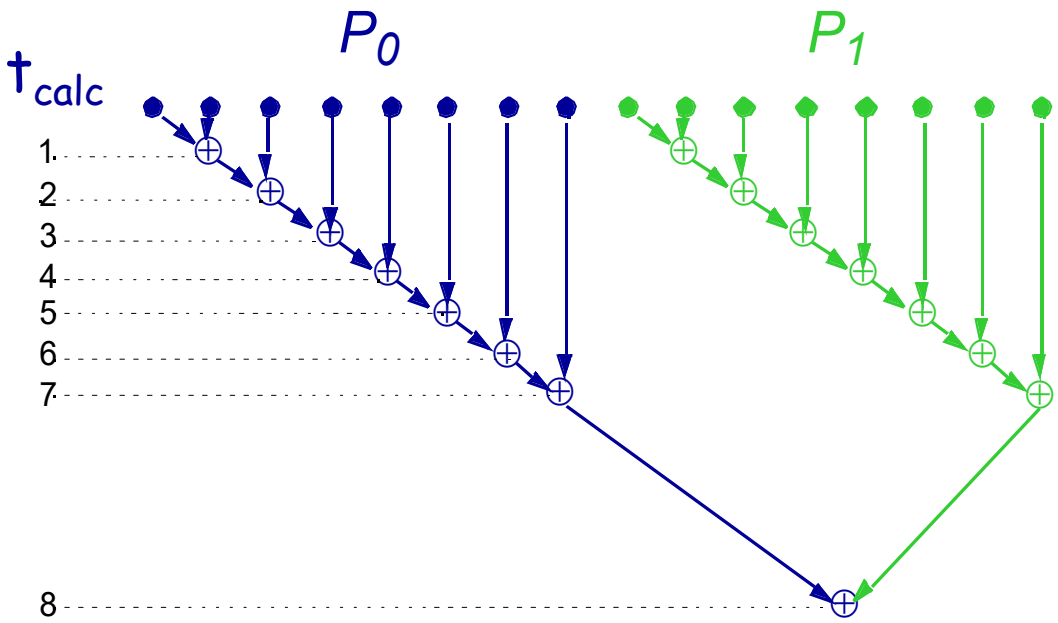
numero di addizioni = 15

$T_1(16)=15$

Esempio: calcolo della somma di N=16 numeri

ALGORITMO PARALLELO

p=2



numero di addizioni = 15

$T_2(16)=8$

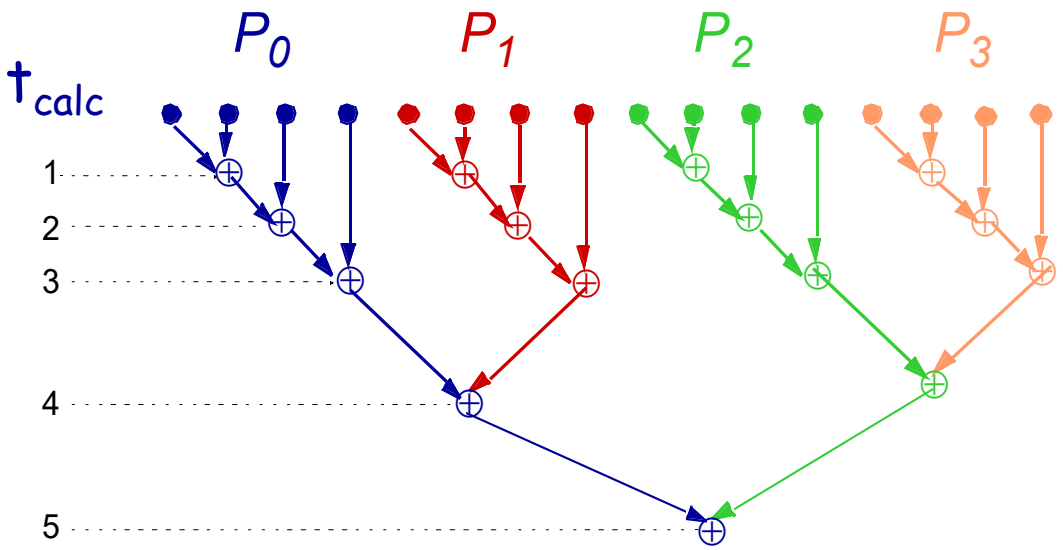
complessità computazionale

2 processori

Esempio: calcolo della somma di N=16 numeri

ALGORITMO PARALLELO

II strategia p=4



numero di addizioni = 15

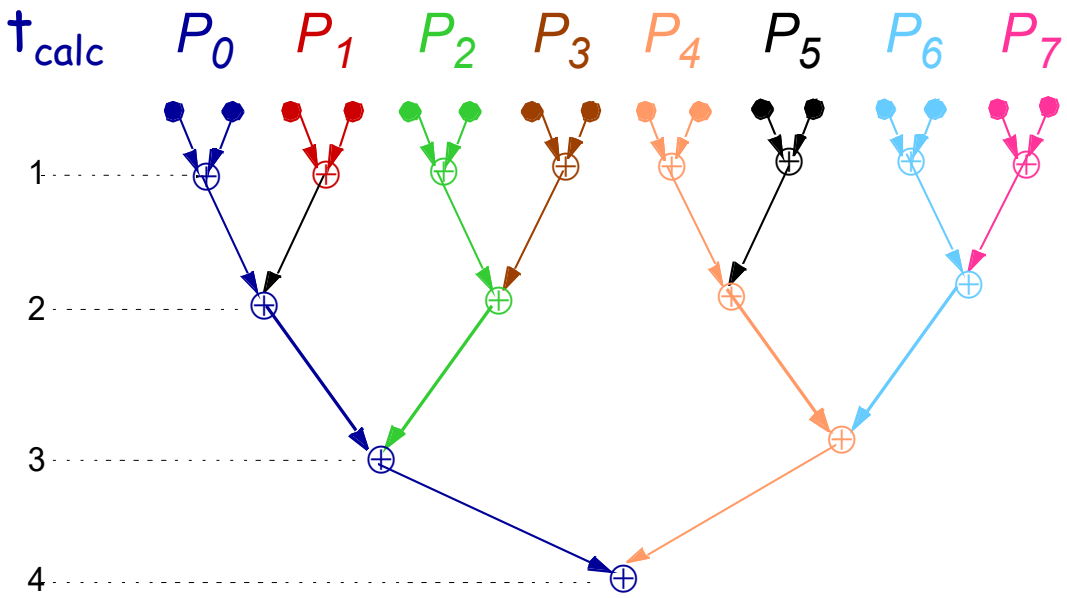
$T_4(16)=5$

complessità computazionale

4 processori

Esempio: calcolo della somma di N=16 numeri

ALGORITMO PARALLELO
II strategia p=8



numero di addizioni = 15

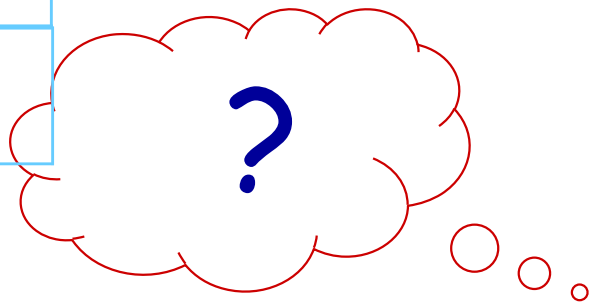
$T_8(16)=4$

complessità computazionale

8 processori

In sintesi... II strategia

p	$T_p(16)$
1	15
2	8
4	5
8	4
p	?



In generale quanto vale T_p?

In generale: calcolo di $T_p(N)$

ALGORITMO PARALLELO della somma di *N* numeri

II strategia - posto p processori

p=1 $T_1=15$

p=2 $T_2=8 = (7+1)$

p=4 $T_4=5 = (3+2)$

p=8 $T_8=4 = (1+3)$

.....

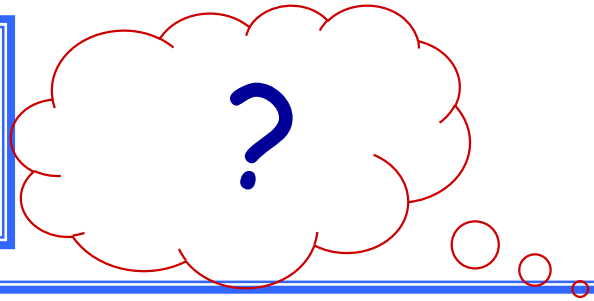
} ***N = 16***

$$T_p(N)= (N/p-1 +\log_2 p)$$

Domanda…

p	T_p
1	15
2	8
4	5
8	4

Qual è l'algoritmo che impiega meno tempo?



Quanto è più veloce di quello sequenziale?

Esempio: calcolo della somma di N=16 numeri

p	T_p	T_1/T_p
1	15	1.00
2	8	1.88
4	5	3.00
8	4	3.75

Maggiore riduzione del tempo ovvero maggiore aumento della velocità

L'algoritmo su 8 processori è il più veloce
E' più veloce di 3.75 volte di quello su 1 processore

Speed-up

Si definisce il rapporto T_1 su T_p

$$S_p = \frac{T_1}{T_p}$$

Lo speed up misura la riduzione del tempo di esecuzione rispetto all'algoritmo su 1 processore

$$S_p < p$$

SPEEDUP IDEALE

$$S_p^{ideale} = p$$

Osservazione

$$S_p^{ideale} = \frac{T_1}{T_p} = p$$

$O_h = (pT_p - T_1)$ → $T_p = (O_h + T_1)/p$

OVERHEAD totale

$$S_p = \frac{T_1}{T_p} = \frac{T_1}{(O_h + T_1)/p} = \frac{pT_1}{O_h + T_1} = \frac{p}{\frac{O_h}{T_1} + 1}$$

Formula piu generica dello speed-up:

L'overhead nell'algoritmo parallelo della somma

II str

$T_1 = (N-1)$

$T_p = (N/p - 1 + \log_2 p)$

$O_h = p\,T_p - T_1 = [p\,(N/p - 1 + \log_2 p) - (N-1)] =$

$= (N - p + p \log_2 p - N + 1)$ $= p \log_2 p - p$

p	O_h
2	2
4	8
8	24
2^k	$p \log_2 p - p$

Al crescere di p l'overhead aumenta!

Quindi

Se si vuole calcolare la somma di 16 numeri nel minor tempo possibile

l'algoritmo su 8 processori è da preferire

Infatti, aumentando il numero di processori si riduce

il tempo impiegato per eseguire le operazioni richieste

QUINDI….

Esempio: calcolo della somma di N=16 numeri

p	Speed-up ottenuto	Speed-up ideale
2	1.88	2
4	3.00	4
8	3.75	8

Lo speed-up su 8 processori è il maggiore

MA

Lo speed-up su 2 processori è "il più vicino" allo speed-up ideale...

Cioè

Ho utilizzato 8 processori per avere un incremento di quasi 4 volte

In altre parole

… lo speed up non basta a fornire informazioni sull'efficienza dell'algoritmo parallelo!

… e allora ?

Esempio: calcolo della somma di N=16 numeri

... se si rapporta lo speed-up al numero di processori...

p	S_p	S_p/p
2	1.88	0.94
4	3.00	0.75
8	3.75	0.47

Rapporto più grande



maggiore sfruttamento dei processori per p=2

In altre parole

l'utilizzo di un maggior numero di processori/core NON è sempre una garanzia di sviluppo di algoritmi paralleli "efficaci"

OVVERO

di algoritmi che sfruttano

tutte le risorse della macchina parallela !

Come misurare se e quanto

è stata sfruttato il calcolatore parallelo ?

L'efficienza di un algoritmo parallelo

Si definisce il rapporto E_p su p

$$E_p = \frac{S_p}{p}$$

misura quanto l'algoritmo sfrutta il parallelismo del calcolatore

EFFICIENZA IDEALE

$$E_p^{ideale} = \frac{S_p^{ideale}}{p} = 1$$